Placement Portal Application

Project Report: Modern Application Development I

IIT Madras Online Degree Programme

1. Author

Full Name	Priyanshi Dalwadi
Roll Number	24F3001002
Email	24f3001002@ds.study.iitm.ac.in
About	I'm a dual degree student (BTech CSE), interested in understanding how systems work at a deeper level and building solutions that are actually useful, not just theoretical. This project is an attempt to apply what I've learned in a more practical, hands-on way and make the concepts feel concrete.

2. AI / LLM Usage

AI tools were used at various stages of this project. Claude (Anthropic) was used to help debug SQLAlchemy queries, understand Flask blueprint structure, and assist with writing parts of this report. DiagramGPT (eraser.io) was used to generate the ER diagram of the database. All AI-generated content was reviewed and adapted manually - database design, route logic, and overall project structure were independently decided and implemented.

Tools used: Claude (Anthropic), DiagramGPT by eraser.io

Category	Backend (Flask + DB)	Frontend (HTML/CSS)	Styling / UI	Auth + CRUD Logic	Extras (Diagrams etc.)
AI Usage	12-17%	15-25%	-	5-8%	35%
Notes	*Used for debugging SQLAlchemy queries and route structure*	*Used for Jinja2 template structure and form layouts, mostly syntax help*	-	*Logic independently written; AI used only for syntax help*	*DiagramGPT used for ER diagram generation*

3. Project Description

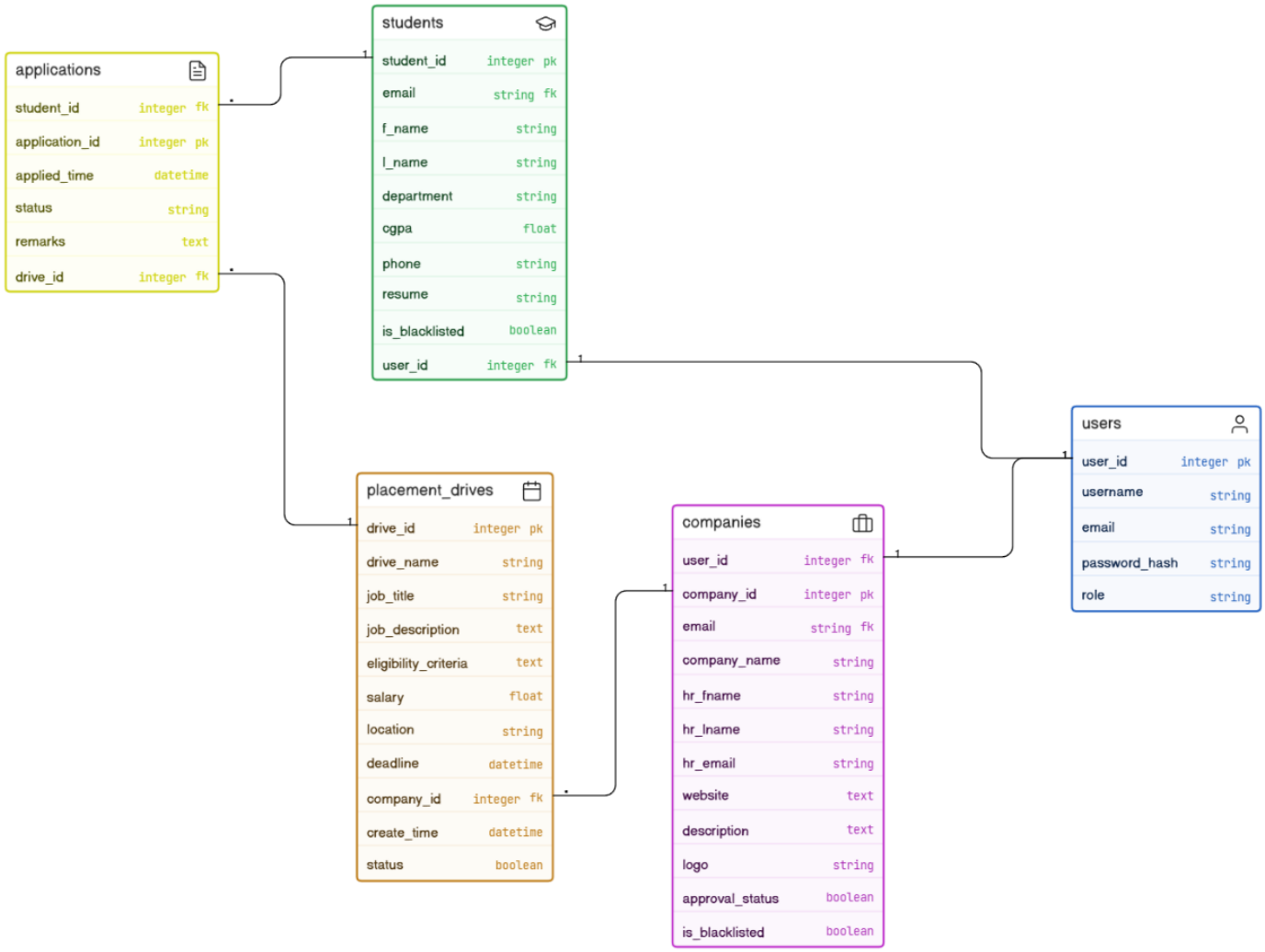
This project is a role-based campus placement portal built using Flask. It allows three types of users — Admin (Institute), Companies, and Students - to interact with the system according to their roles.

- Companies can register, post placement drives, and manage applications.
- Students can browse active drives and apply to them.
- The Admin oversees the entire system: approving companies, managing drives, and monitoring student and application data.

4. Technologies Used

Technology	Purpose
Python 3 / Flask	Core web framework — handles routing, request/response lifecycle, and blueprint-based modular structure
Flask-SQLAlchemy	ORM layer for database interactions; maps Python model classes to SQLite tables
SQLite	Lightweight file-based relational database; suitable for development and local deployment
Werkzeug	Password hashing (generate_password_hash / check_password_hash) and secure filename handling for uploads
Jinja2	Server-side HTML templating; renders dynamic content in all views
Flask Sessions	Stateless server-side sessions used to track logged-in user identity and role across requests
HTML / CSS	Frontend presentation layer via Jinja2 templates, organised by role (auth, admin, company, student)

5. Database Schema Design



7. Architecture and Features

Architecture

```
placement-portal-app/
├── app.py
├── models.py
├── extensions.py
├── routes/
│   ├── auth.py
│   ├── admin.py
│   ├── company.py
│   └── student.py
├── templates/
│   ├── auth/
│   ├── admin/
│   ├── company/
│   └── student/
└── static/
    └── uploads/
        ├── resumes/
        └── logos/
```

The project follows a standard MVC-style Flask application structure. app.py is the entry point: it creates the Flask application, registers four Blueprints (auth, admin, company, student), initialises SQLAlchemy via extensions.py, and seeds a default admin user on first run. Business logic and route handlers live in the routes/ directory, one file per role. Database models are defined in models.py using SQLAlchemy declarative syntax. All HTML views are Jinja2 templates stored under templates/, grouped by role (auth/, admin/, company/, student/). Static files — uploaded resumes and logos — are stored under static/uploads/.

Default Features

- Role-based authentication with session management (admin, company, student)
- Company registration with admin approval gate before login is permitted
- Placement drive lifecycle: create → active → closed (manual or automatic on deadline)
- Student application flow: browse drives → apply → view status (waiting / shortlist / reject)
- Admin dashboard showing aggregate stats: total students, approved companies, drives, and placed count
- File uploads: PDF resumes for students, PNG/JPG logos for companies
- Search and filter on admin management views (companies, drives, students)

Additional Features

- Automatic drive expiry: on every dashboard load, drives past their deadline are automatically marked as closed
- Blacklisting: admin can toggle blacklist status on both companies and students to restrict access
- Company ownership enforcement: companies can only edit or close their own drives; unauthorised attempts are rejected with a flash message
- Resume access control: student resumes are viewable by the company reviewing the application and by the admin
- Application remarks: companies can leave private notes alongside shortlist/reject decisions

8. Video

https://drive.google.com/drive/folders/1cJucKnLy5rA_CxLSY2bgCZ2gmqoZg8uf?usp=sharing